

CS 621 HW 4

Ethan Reinhart

November 2025

1

Problem

Let $T = (V, E)$ be a rooted tree. Define a pedigree graph $G = (V, E')$ of T as a directed graph where an edge (u, v) is in E' if and only if $v \neq u$ is a descendant of u in T . Suppose you are given only the pedigree graph G of T . Provide an algorithm reconstructing the original T .

Solution

For any vertex v in T :

- The ancestors of v are exactly the in-neighbors of v in G :

$$A(v) := \{u \in V : (u, v) \in E'\}.$$

- In a rooted tree, the ancestors of v lie on a single root-to- v path, so $A(v)$ is totally ordered by the ancestor relation.
- The parent $p(v)$ is the ancestor of v that is closest to v , i.e. the unique maximal element of $A(v)$ under the ancestor relation.

In G , for any two distinct ancestors $x, y \in A(v)$, one of them is an ancestor of the other, so exactly one of (x, y) or (y, x) is in E' . Thus, among vertices in $A(v)$, the parent $p(v)$ is exactly the vertex u such that

$$(w, u) \in E' \quad \text{for all } w \in A(v) \setminus \{u\},$$

i.e. u has incoming edges from all the other ancestors of v .

Assume we have access to G via adjacency lists of outgoing edges and also maintain for each vertex its list of incoming neighbors (we can build the in-neighbor lists in $O(|V| + |E'|)$ time):

1. For each vertex $v \in V$, compute its ancestor set

$$A(v) := \{u : (u, v) \in E'\}$$

from the incoming adjacency lists.

2. If $A(v) = \emptyset$, then v is the *root* of T (there is a unique such vertex).

3. Otherwise, find the parent of v as follows:

- Initialize p to any element of $A(v)$.
- For each $w \in A(v)$:
 - If $w = p$, continue.
 - If $(w, p) \in E'$, then w is an ancestor of p , so p is deeper; keep p unchanged.
 - If $(p, w) \in E'$, then p is an ancestor of w (i.e. shallower than w), so set $p \leftarrow w$.
- After processing all $w \in A(v)$, the current p is the unique maximal ancestor of v , i.e. the parent $p(v)$.

4. Output the tree $T = (V, E)$ where

$$E = \{(p(v), v) : v \in V, A(v) \neq \emptyset\}.$$

This algorithm is correct for a few reasons:

- For every v , $A(v)$ contains exactly all ancestors of v in T by the definition of G .
- In a rooted tree, ancestors of v form a chain, so $A(v)$ is totally ordered by ancestry. Hence, there is a unique maximal element in $A(v)$, the parent $p(v)$.
- In the algorithm, whenever we compare p and w :
 - If $(w, p) \in E'$, then w is above p , so p cannot be the parent of v (it is too high in the tree), and we do *not* replace p .
 - If $(p, w) \in E'$, then p is above w , so w is a lower ancestor; we update p to w .

This is exactly the standard “maximum in a total order by comparisons” process.

- Thus, we correctly recover $p(v)$ for every $v \neq \text{root}$, and hence reconstruct the original tree T .

2

Problem

Consider an undirected graph G with n vertices and m edges. Given an adjacency list of G as input, provide an algorithm checking whether G is a tree with an $O(m)$ average-case time complexity. You should also justify your complexity.

Solution

If G is a tree, then its unique connected component contains all n vertices and has exactly $n - 1$ edges. If there are extra vertices or extra edges, we will detect either a cycle or that not all vertices were visited.

Here is the algorithm:

1. Choose any vertex $r \in V$ as the root of a search (e.g. the first vertex in the adjacency list).

2. Initialize:

- A hash set $Visited := \emptyset$ to store visited vertices.
- A hash map $Parent$ to store the parent of each visited vertex in the DFS/BFS tree.
- A counter $edge_count := 0$.

3. Perform a DFS or BFS starting from r :

- Insert r into $Visited$, set $Parent[r] := \text{null}$.
- While the search frontier is not empty:
 - Remove a vertex u from the frontier.
 - For each neighbor v in the adjacency list of u :
 - * If $v \notin Visited$:
 - Insert v into $Visited$.
 - Set $Parent[v] := u$.
 - Add v to the frontier.
 - Increment $edge_count$ by 1 (we have just discovered a new undirected edge $\{u, v\}$).
 - * Else if $v \in Visited$ and $v \neq Parent[u]$:
 - We have found a back edge $\{u, v\}$, hence a cycle. Return “ G is not a tree”.

4. After the search finishes, let $k := |Visited|$.

5. If $k \neq n$, then G is disconnected (there are vertices not in the component of r), so return “ G is not a tree”.

6. If $edge_count \neq n - 1$, then G does not have the right number of edges (too few or too many), so return “ G is not a tree”.

7. Otherwise, G is connected and acyclic with $n - 1$ edges, so return “ G is a tree”.

The search visits each vertex in the connected component of r at most once and inspects each incident edge at most a constant number of times.

Thus the search takes $O(m)$ time in that component. We never explicitly initialize an array of size n : we use a hash set for $Visited$ and a hash map for $Parent$.

Each hash-table operation is $O(1)$ average case, so the total hash overhead is $O(m)$ on average. If G is a tree, then it is connected and has $m = n - 1$, so the DFS/BFS explores all vertices and edges and still runs in $O(m)$ time.

If G is not a tree:

- Either the component of r has a cycle, which we detect during the search without visiting the whole graph, or
- The component of r is acyclic but does not contain all vertices; then $k < n$ and we detect disconnectedness at the end. The time is still proportional to the size of the explored component, hence $O(m')$ where $m' \leq m$.

Thus, in all cases, the algorithm runs in $O(m)$ time on average, using hash maps/sets to avoid any $O(n)$ preprocessing.